1. Параметрическая и не параметрическая оценка

не парам. оценка для распределения →
→ выборочная ф.-ия распределения (гистограмма)
(теорема Гливенко-Кантелли)

Пусть имеем $X_1, \dots, X_n \sim N(a, \sigma^2)$
a, σ^2 – параметры (достаточные статистики)
абсолютное значение среднего не статистика, потому что полностью не задаёт распределения

для линейной модели
$f_\theta(x) = w^T x + b$, $\theta = (\vec{w}, b)$ – мн.-во параметров

Разница между параметром и гиперпараметром?

① Парам. взаимодействуют с объектами.
например: $f_\theta(x) = \boxed{w^T} \cdot x + \boxed{b}$ напрямую,

② а гиперпараметры взаимодействуют с моделью. Пока мы не определим гиперпараметр модель не существует. Она не определена.
например: knn $(k = ?)$

③ Необх. аргументы для построения конструктора, а параметры для методов модели.

1. KNN
2. Naive Bayes
3. Linear
4. Trees
5. Ensembles

7. NN 🧠⚡

$$P(A|B) = \frac{P(B|A)\cdot P(A)}{P(B)}$$

Linear
logistic
SVM

$$w^T \cdot x + b$$

выбор ядра

$k(x', x) \longrightarrow$ target

Кусочно постоянная функция, которая берёт всё наше пространство и делит на какие-то непересекающиеся (как правило) области и для каждой области сопоставляем константу. (можно и функцию, чтоб получить линейную функцию или что-нибудь ещё)

$J_i \Big|_{i=1}^{k}$ — множество листьев

$$\hat{y} = \sum_{i=1}^{k} w_i \cdot \underbrace{I\{x \in J_i\}}, \text{ где } w_i \text{ — константы}$$

Это взвешанная сумма "новых признаков"

stacking
blending
bagging
boosting

Random Forest

$$\hat{y} = \text{Signal} + \varepsilon$$

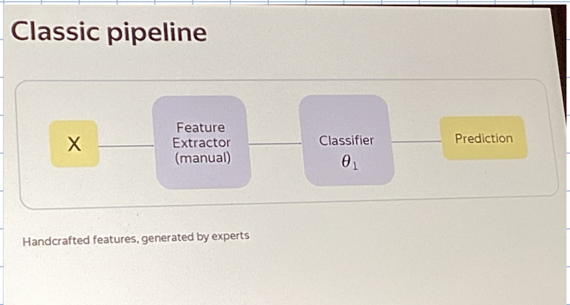
ε ↓ случайная величина

шум (noise)

$$g(x) = \sum_i \alpha_i \cdot f_i(x)$$ (сложим все модели и собираем в одну)

$$X_0 \to X_1 \to X_2 \to \dots \to Y$$

? А какие преобразования нам брать между нашими пространствами, чтобы получить хороший результат? Линейные, ... преобраз.

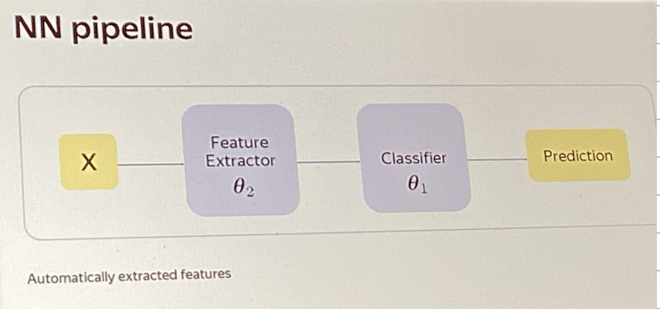


Classic pipeline

X
Feature Extractor (manual)
Classifier θ_1
Prediction

Handcrafted features, generated by experts

→

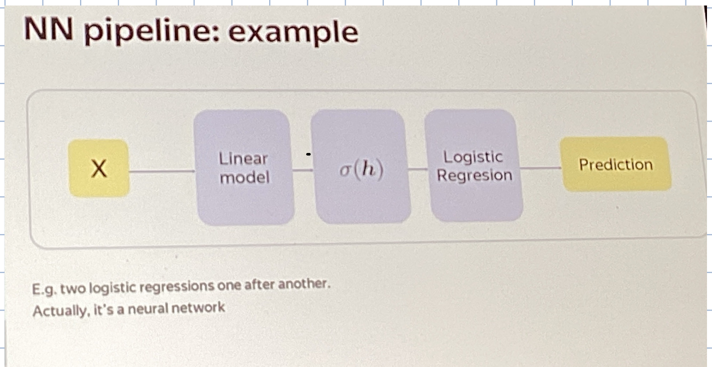


NN pipeline

X
Feature Extractor θ_2
Classifier θ_1
Prediction

Automatically extracted features

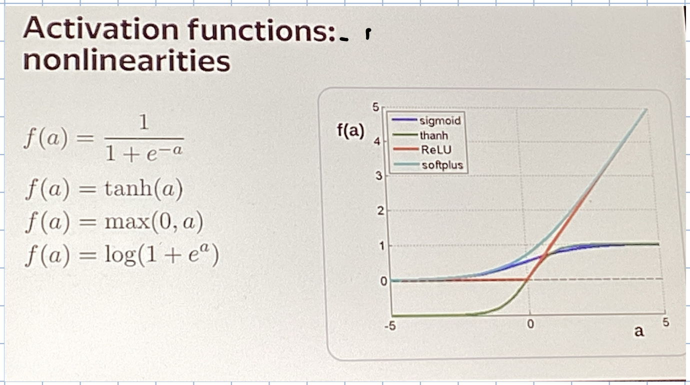
Example:



NN pipeline: example

X — Linear model — $\sigma(h)$ — Logistic Regresion — Prediction

E.g. two logistic regressions one after another.
Actually, it's a neural network



Activation functions: nonlinearities

$$f(a) = \frac{1}{1+e^{-a}}$$

$$f(a) = \tanh(a)$$

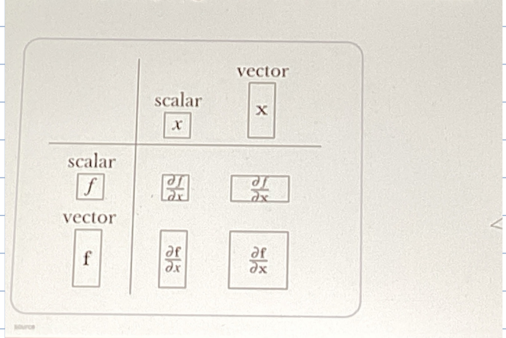
$$f(a) = \max(0, a)$$

$$f(a) = \log(1+e^{a})$$

Свёртка случайных величин

[illegible]

Backpropagation

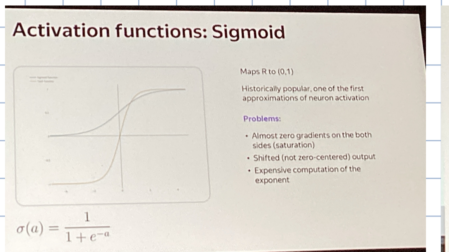


Backpropagation: matrix form

	scalar x	vector $\mathbf{x}$
scalar f	$\frac{\partial f}{\partial x}$	$\frac{\partial f}{\partial \mathbf{x}}$
vector $\mathbf{f}$	$\frac{\partial \mathbf{f}}{\partial x}$	$\frac{\partial \mathbf{f}}{\partial \mathbf{x}}$

source

Activation functions



Activation functions: Sigmoid

Maps R to (0,1)

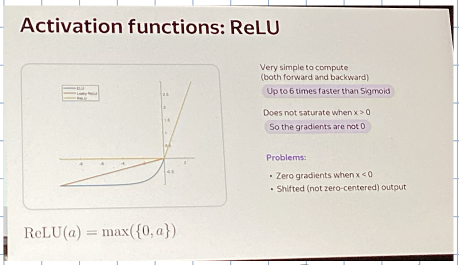
Historically popular, one of the first approximations of neuron activation

Problems:

- Almost zero gradients on the both sides (saturation)
- Shifted (not zero-centered) output
- Expensive computation of the exponent

$$\sigma(a) = \frac{1}{1+e^{-a}}$$

Затухает градиент



Activation functions: ReLU

Very simple to compute
(both forward and backward)

Up to 6 times faster than Sigmoid

Does not saturate when x > 0

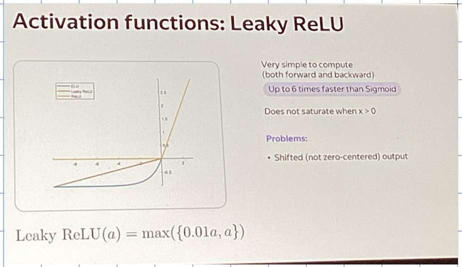
So the gradients are not 0

Problems:

- Zero gradients when x < 0
- Shifted (not zero-centered) output

$$\mathrm{ReLU}(a) = \max(\{0, a\})$$

Кусочно-линейная ф-ия



Activation functions: Leaky ReLU

Very simple to compute
(both forward and backward)

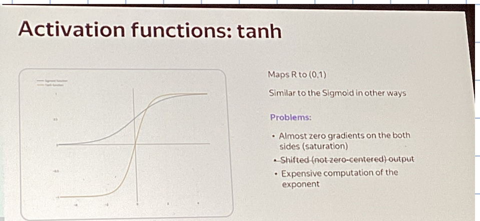
Up to 6 times faster than Sigmoid

Does not saturate when x > 0

Problems:

- Shifted (not zero-centered) output

$$\text{Leaky ReLU}(a) = \max(\{0.01a, a\})$$



Activation functions: tanh

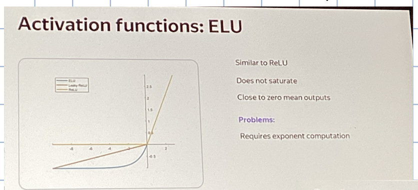
Maps R to (0,1)

Similar to the Sigmoid in other ways

Problems:

- Almost zero gradients on the both sides (saturation)
- ~~Shifted (not zero-centered) output~~
- Expensive computation of the exponent

$$\tanh(a) = \frac{e^{a}-e^{-a}}{e^{a}+e^{-a}}$$



Activation functions: ELU

Similar to ReLU

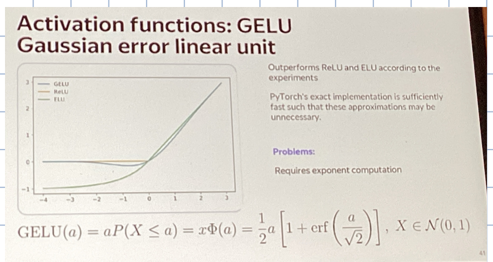
Does not saturate

Close to zero mean outputs

Problems:

Requires exponent computation

$$\mathrm{ELU}(a) = \begin{cases} a & \text{if } a > 0 \\ \beta(\exp(a) - 1) & \text{if } a \le 0 \end{cases}$$



Activation functions: GELU Gaussian error linear unit

Outperforms ReLU and ELU according to the experiments

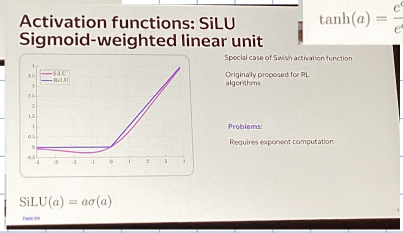
PyTorch's exact implementation is sufficiently fast such that these approximations may be unnecessary.

Problems:

Requires exponent computation

$$\mathrm{GELU}(a) = aP(X \le a) = x\Phi(a) = \frac{1}{2}a\left[1 + \mathrm{erf}\left(\frac{a}{\sqrt{2}}\right)\right],\ X \in \mathcal{N}(0,1)$$

Paper link



Activation functions: SiLU Sigmoid-weighted linear unit

Special case of Swish activation function

Originally proposed for RL algorithms

Problems:

Requires exponent computation

$$\mathrm{SiLU}(a) = a\sigma(a)$$

Paper link

Теорема об универсальном апроксиматоре (читать про это)
universal approximation theorems

Цель модели — это решение той оптимизационной задачи, которое мы перед ней поставили.